

Collection

- list → 1. arraylist 2. list
- set → 1. hashset 2. treeset
- Map → 1. hashmap 2. treemap

1. arraylist

Characteristics

1. allow dynamic allocation
2. allow different datatype
3. allow duplicate values
4. don't have sorting

class sample

```
public static void main(String args)
{
    ArrayList element = new ArrayList();
    element.add(10);
    element.add(20);
    element.add("name");
    for (Object obj : element)
    {
        System.out.println(obj);
    }
}
```

2. list

Characteristics

1. allow dynamic allocation
2. don't allow different datatype
3. allow duplicate values
4. don't have sorting

class sample

```
{  
    public static void main (String [] args)  
    {
```

```
        List <Integer> elements = new List <Integer> ();
```

```
        element.add (10);
```

```
        element.add (20);
```

```
        element.add (30);
```

```
        for (Object obj : elements)
```

```
        {  
            System.out.println (obj);  
        }
```

```
    }  
}
```

3. hashset

Characteristics

1. set means build some algorithms for unordered

2. hash don't allow duplicate values

class sample

```
{  
    public static void main (String [] args)  
    {
```

```
        HashSet <Integer> elements = new HashSet  
        <Integer> ();
```

```
        elements.add (10);
```

```
        elements.add (20);
```

```
        element.add (30);
```

for (Object

```
{  
    System.out
```

```
}
```

```
}
```

4. TreeSet

Characteristics

1. allow duplicate
2. don't allow
3. don't allow
4. allow

class sample

```
{  
    public static
```

```
{
```

TreeSet <Integer>

element

element

element

for (Object

```
{
```

System.out

```
}
```

```
}
```

```
}
```

5. HashMap

Characteristics

1. hash don't
2. don't


```

for (Object ob : elements)
{
    System.out.println(ob);
}
}
}

```

4. TreeSet

Characteristics

1. allow dynamic allocation
2. don't allow different datatype
3. don't allow duplicate values
4. allow have sorting

class sample

```

{
    public static void main (String [] args)
    {
        TreeSet<Integer> element = new TreeSet<Integer>
        ();
        element.add(10);
        element.add(30);
        element.add(20);
        for (Object ob : element)
        {
            System.out.println(ob);
        }
    }
}

```

5. HashMap

Characteristics

1. hash don't allow duplicate values
2. don't allow sorting

4. use Key and values in datatype

class sample

{

public static void main (String [] args)

{

HashMap<Integer, String> element = new HashMap

<Integer, String>

element.put (10, "a");

element.put (20, "b");

for (Map.Entry obj : element.entrySet())

{

System.out.println (obj.getKey() + " " +

obj.getValue());

}

6. TreeMap

1. allow sorting

2. don't allow duplicate value

3. use key and value in datatype

class sample

{

public static void main (String [] args)

{

TreeMap<Integer, String> element = new TreeMap

<Integer, String>();


```
element.put(20, "a");
```

```
element.put(10, "b");
```

```
for (Map.Entry obj : element.entrySet())
```

```
    System.out.println(obj.getKey() + " +
```

```
    obj.getValue());
```

```
}
```

```
}
```

```
}
```